R Notebook

Code

##1.2.5

Hide

```
library(tidyverse)
library(palmerpenguins)
library(ggthemes) #provides colour blind safe colour palette

view(penguins)
```

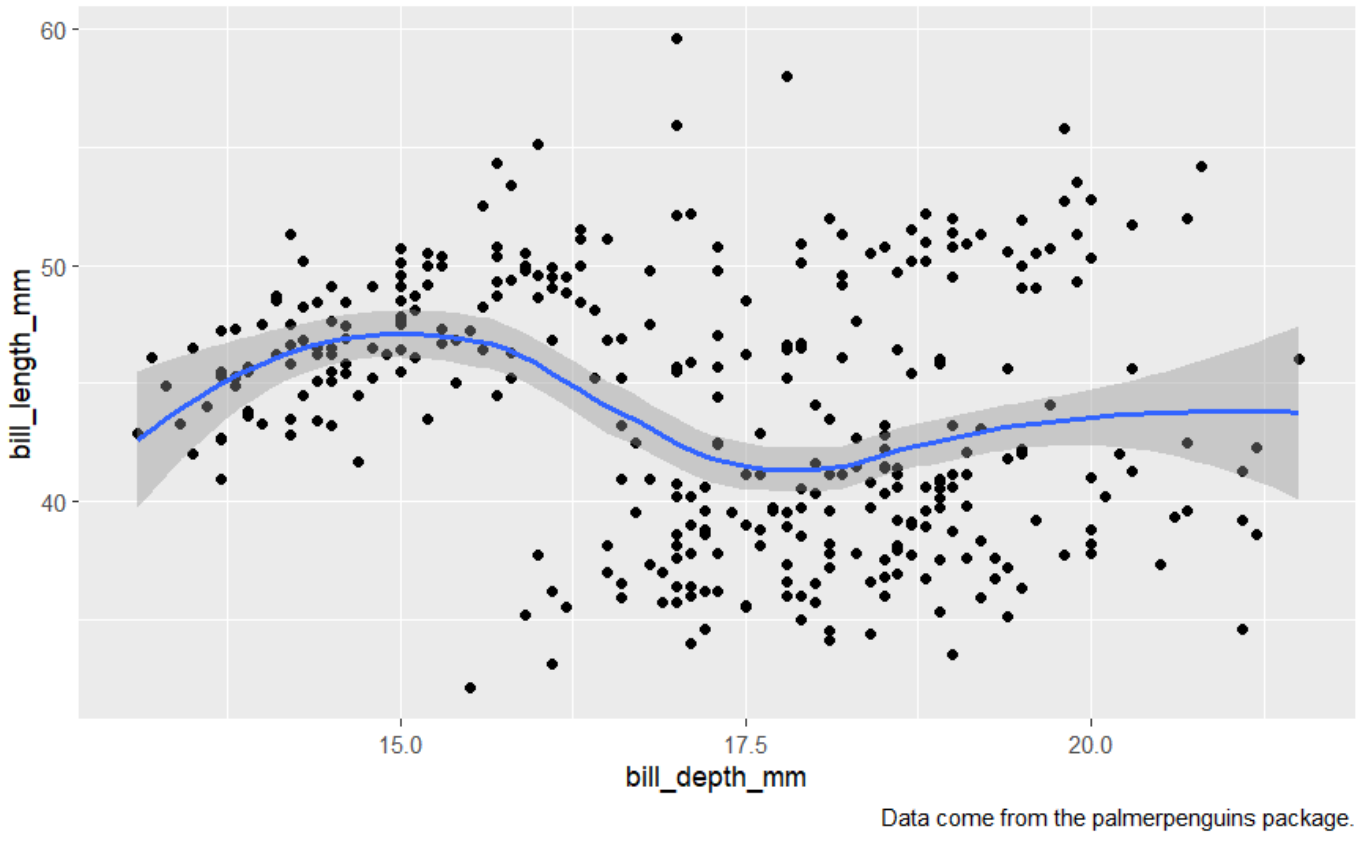
#7

Using labs() we can add caption as shown below.

Hide

```
ggplot(
  data = penguins,
  mapping = aes(x = bill_depth_mm, y = bill_length_mm)

)+
  geom_point(na.rm = TRUE)+
  geom_smooth()+
  labs(
    caption = "Data come from the palmerpenguins package."
  )
```



Hide

```
NA
NA
```

#8

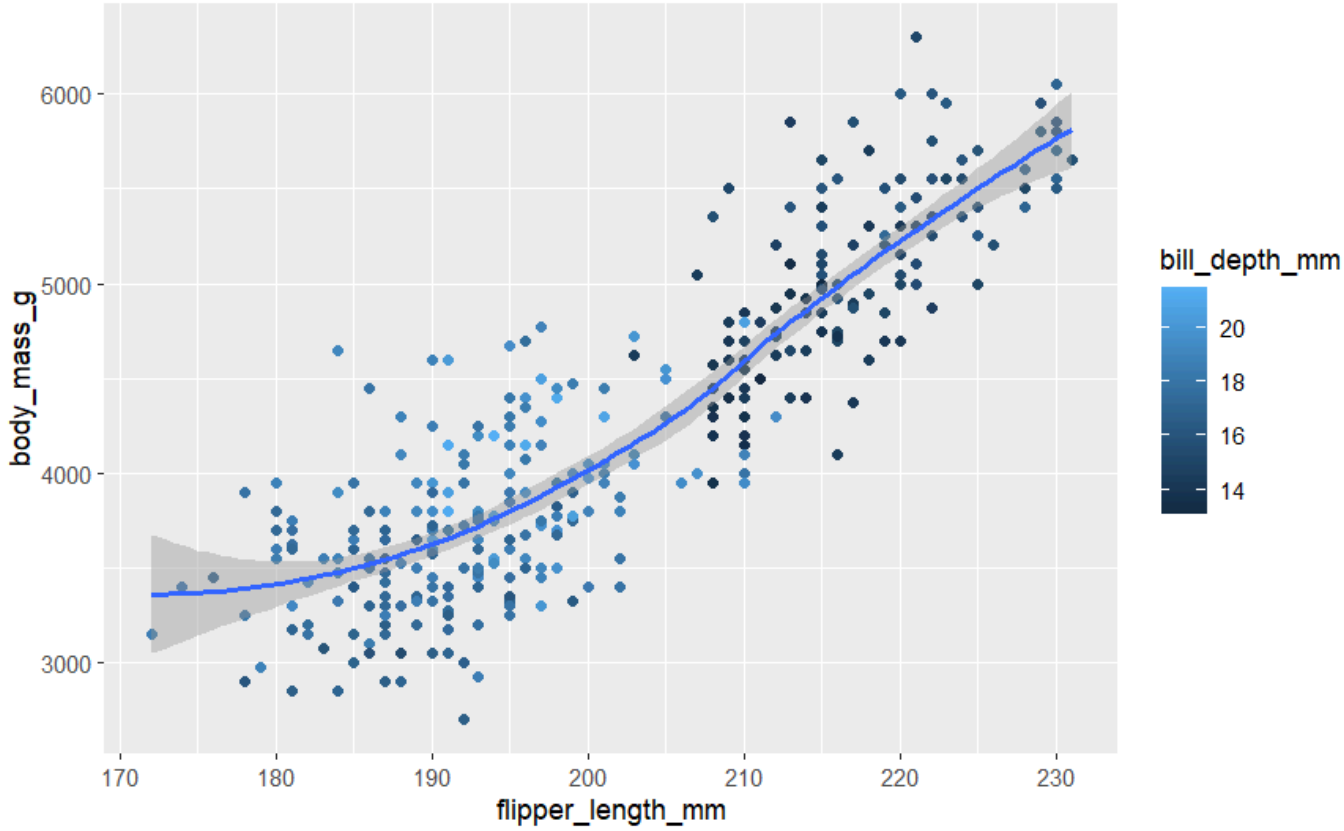
bill_depth_mm should be mapped to color aesthetic as shown above. It should be mapped to geom level such that it is mapped continuously to the color aesthetic. If we map this to a global level then both points and smooth line would try to vary by bill_depth_mm.

Hide

```
ggplot(

  data = penguins,
  mapping = aes(x = flipper_length_mm, y = body_mass_g)

)+
  geom_point(aes(color = bill_depth_mm), na.rm = TRUE)+
  geom_smooth()
```



Hide

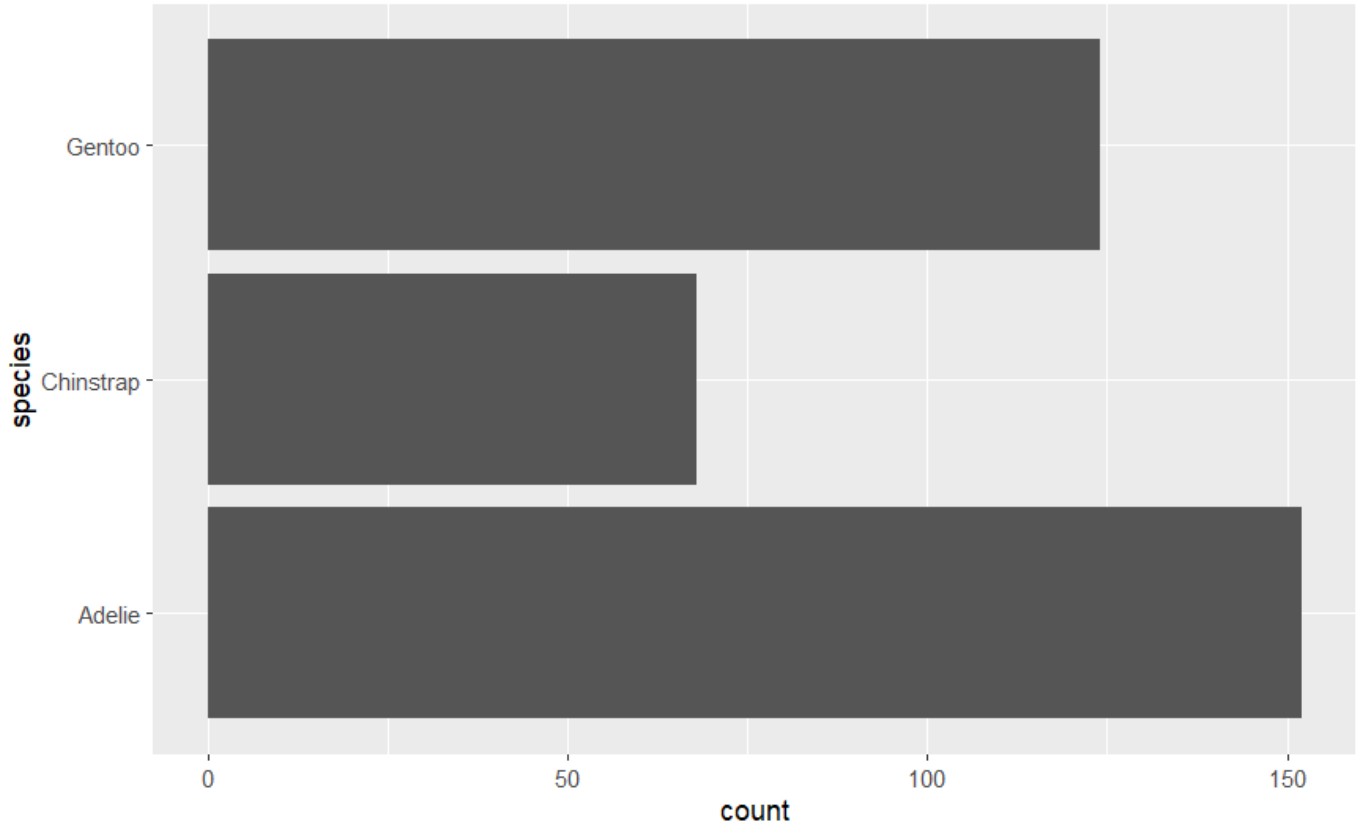
```
NA
NA
```

1.4.3

#1 On making this plot assigned to y aesthetic it is positioned according to the y axis which is in landscape and not potrait.

Hide

```
ggplot(penguins, aes(y = species))+
  geom_bar()
```

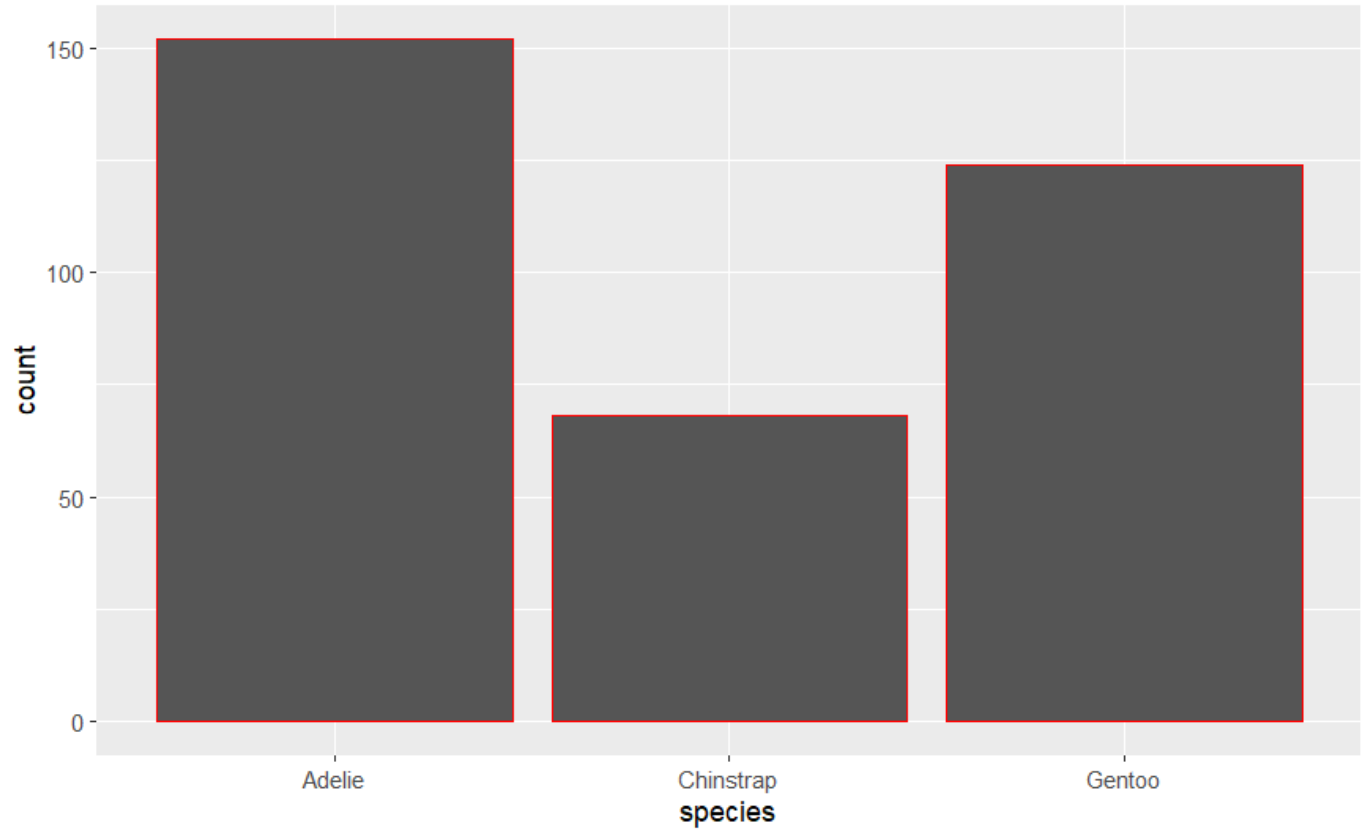


#2

so the first graph just has a very light border of red color and it does not show much significant difference from the normal graph. However, the second graph has its bar coloured entirely red which makes it different and stand out from the previous graph. So the color might just colour the outer sides of the bar plots where as the fill makes sure that the bar plots are filled in that particular colour.

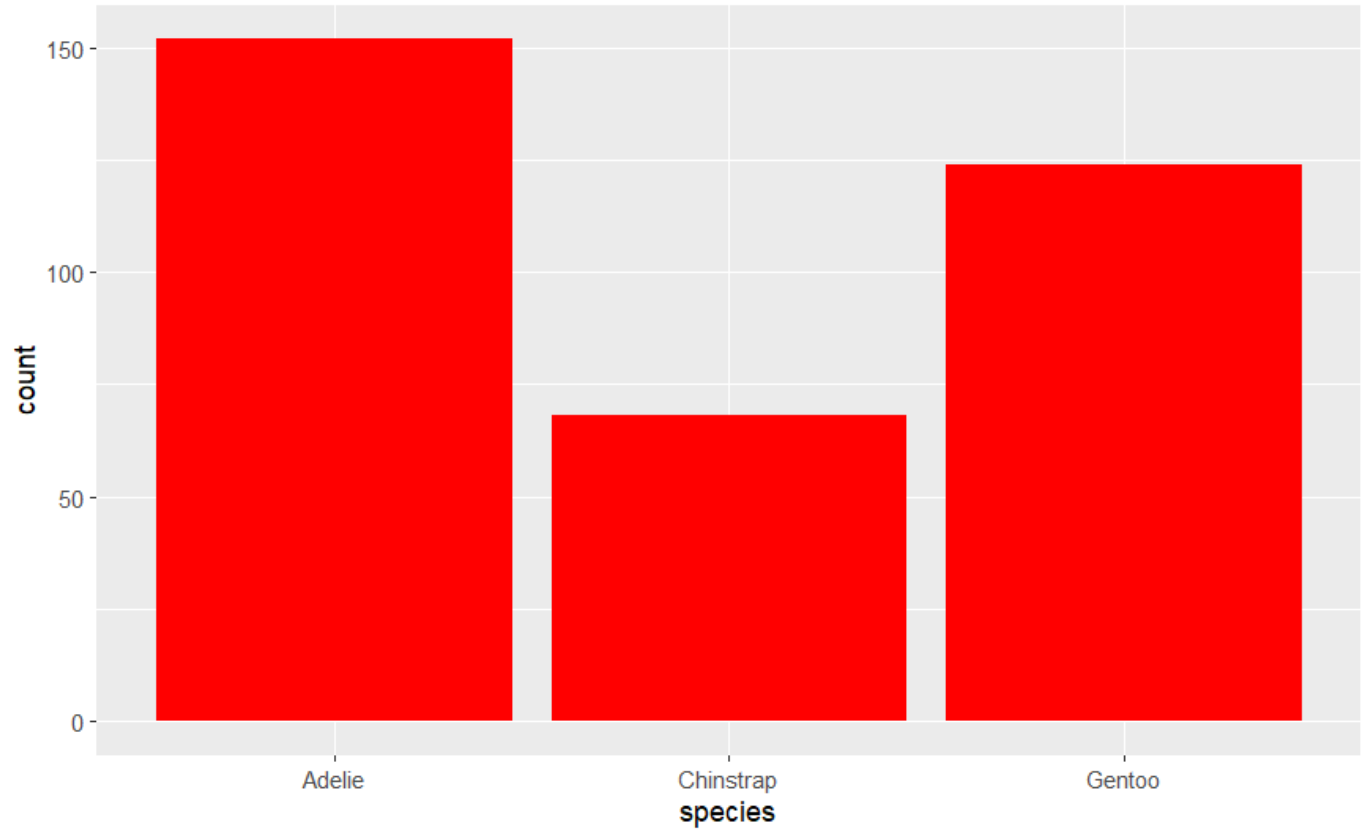
Hide

```
ggplot(penguins, aes(x = species)) +
  geom_bar(color = "red")
```



Hide

```
ggplot(penguins, aes(x = species)) +
  geom_bar(fill = "red")
```

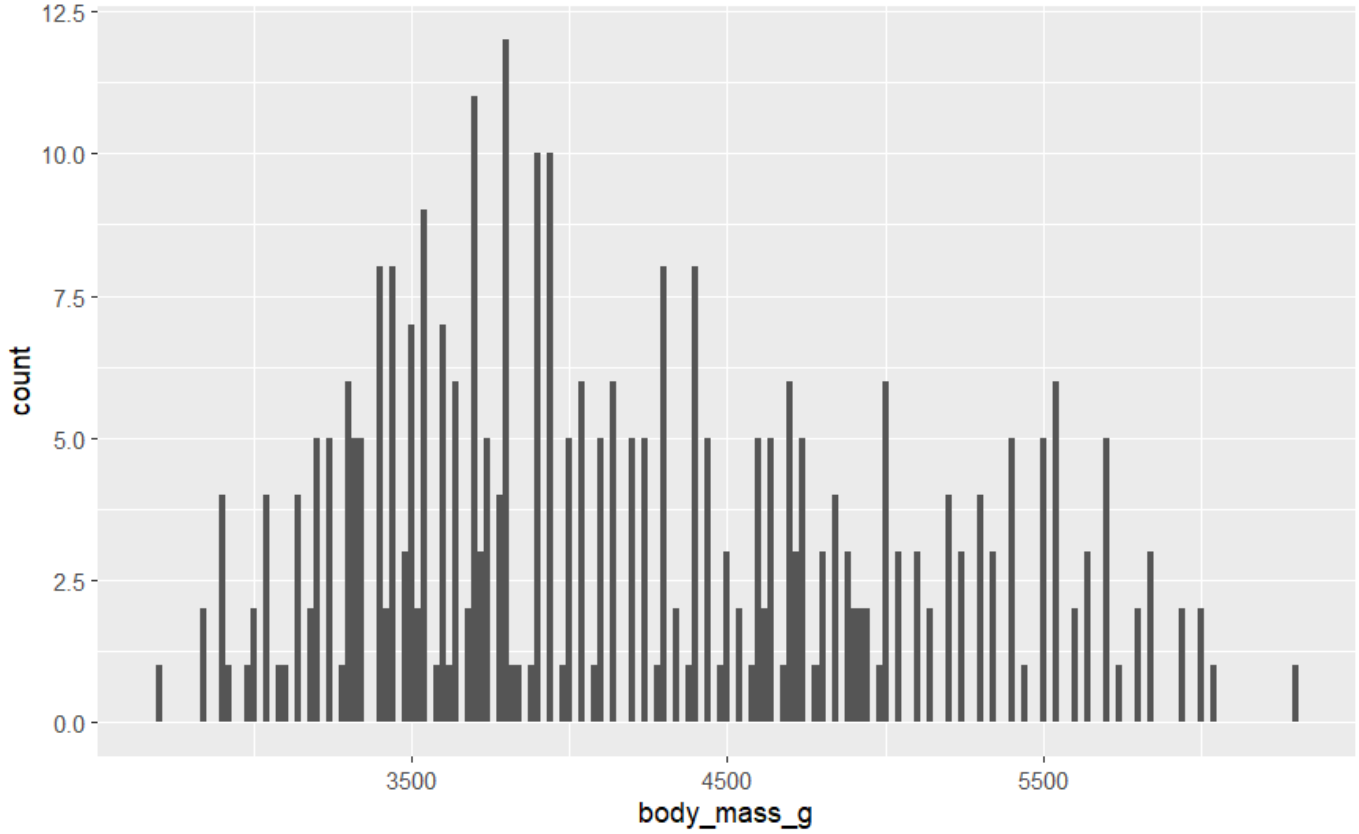


#3 the bins arguement in geom_histogram() equally divides the x-axis into equally apced bins making the vizalisation more tidy and easy to read. This then uses the height of a bar to display the number of observations that fall in each bin. As shown below:

The histogram has its x-axis equally spaced with a binwidth of 20 (this measures the units of x variable.

Hide

```
ggplot(penguins, aes(x = body_mass_g))+
  geom_histogram(binwidth = 20)
```

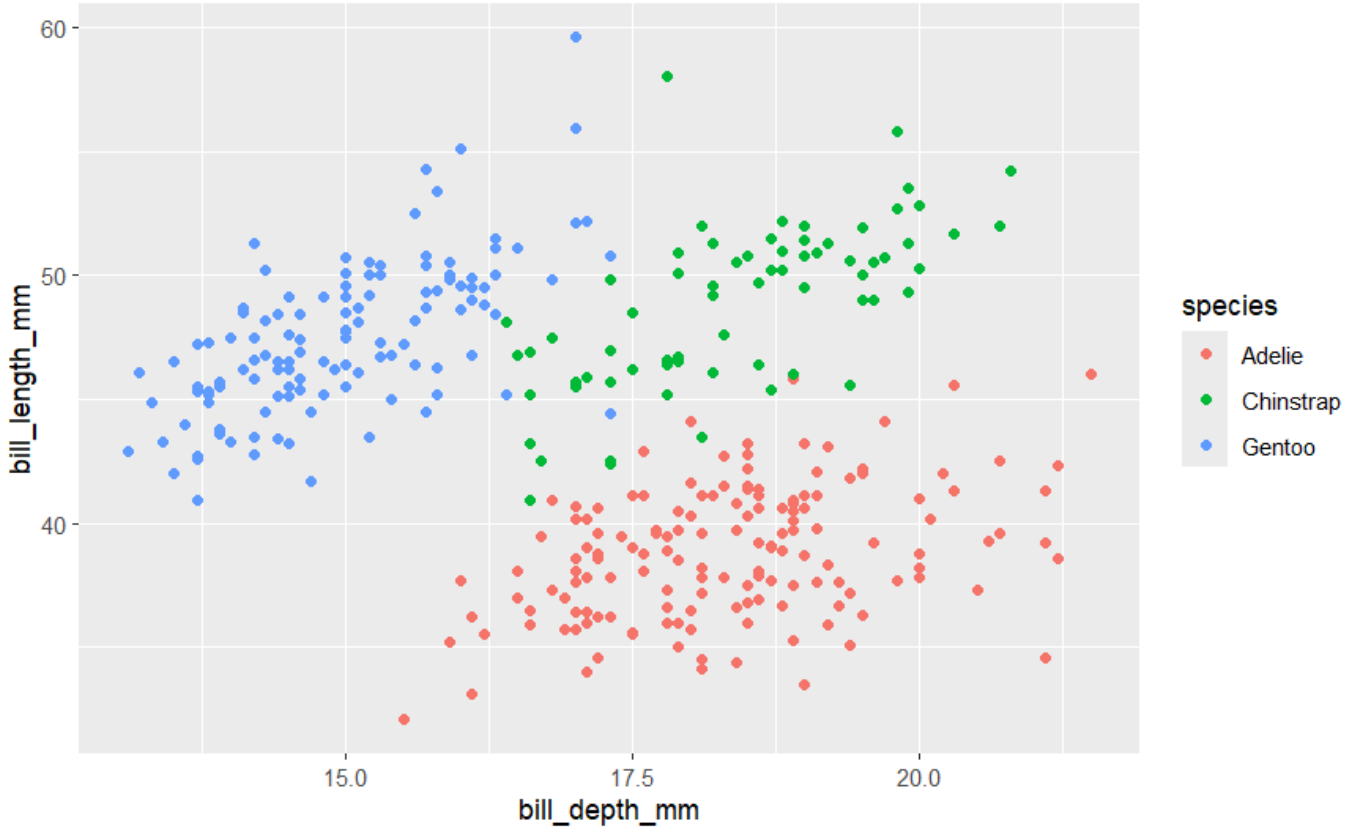


##1.5.5

#5 This plot shows that Adeline tend to have shorter bills with moderate depth. Chinstraps tend to have longer bills with similar depths and Gentoo shorter, shallower bills altogether. This graph shows between-spaces separation mpre than a strong within-species correlation.

Hide

```
ggplot(penguins, aes(x = bill_depth_mm, y = bill_length_mm))+
  geom_point(aes(color = species))
```

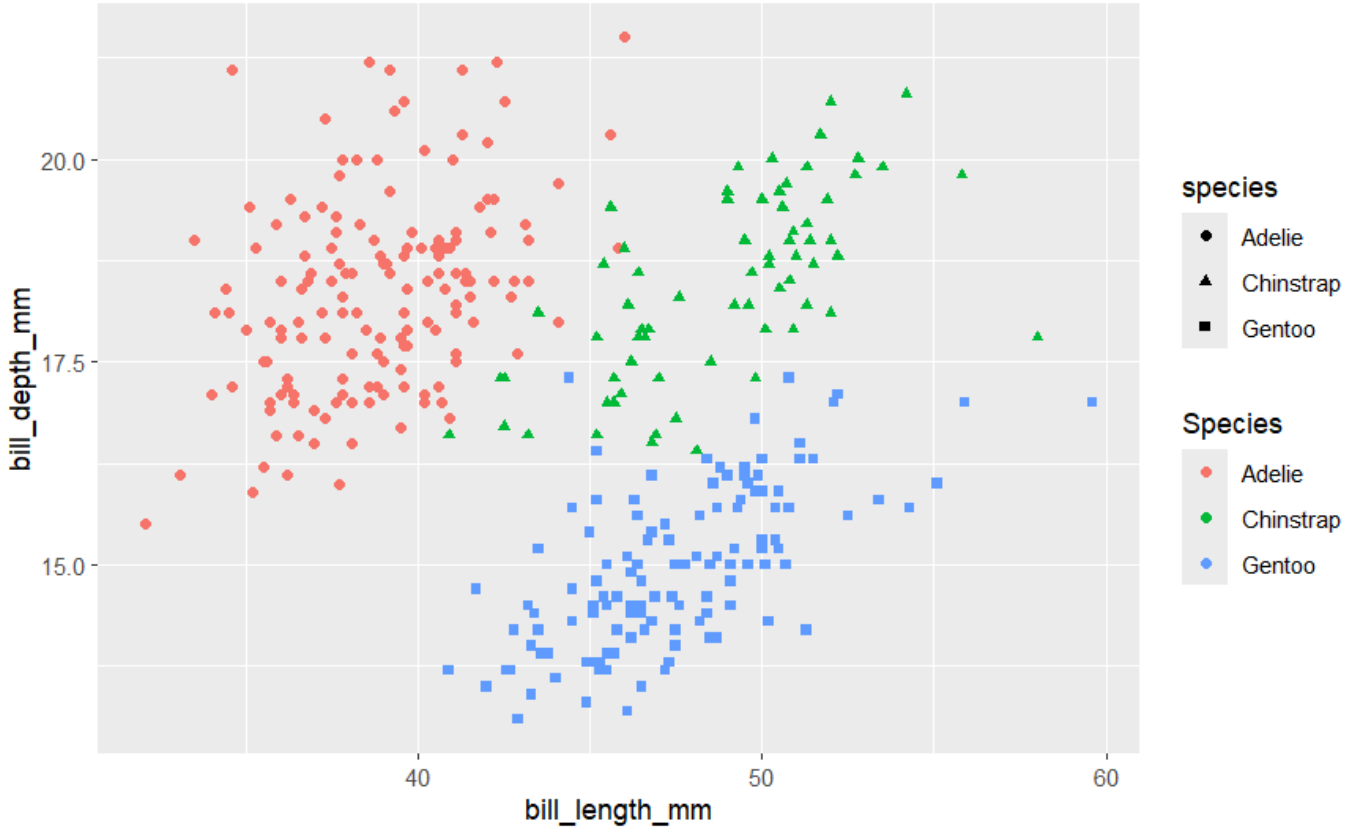


#6

This graph yields 2 legends because we are plotting the graph to be bill_length_mm vs bill_depth_mm making sure that the color points aesthetic are along species. on doing this we are already defining a legend but later on in this code there is again a repetition of labs() defining color aesthetic resulting in 2 legends.

Hide

```
ggplot(
  data = penguins,
  mapping = aes(
    x = bill_length_mm, y = bill_depth_mm,
    color = species, shape = species
  )
) +
  geom_point() +
  labs(color = "Species")
```

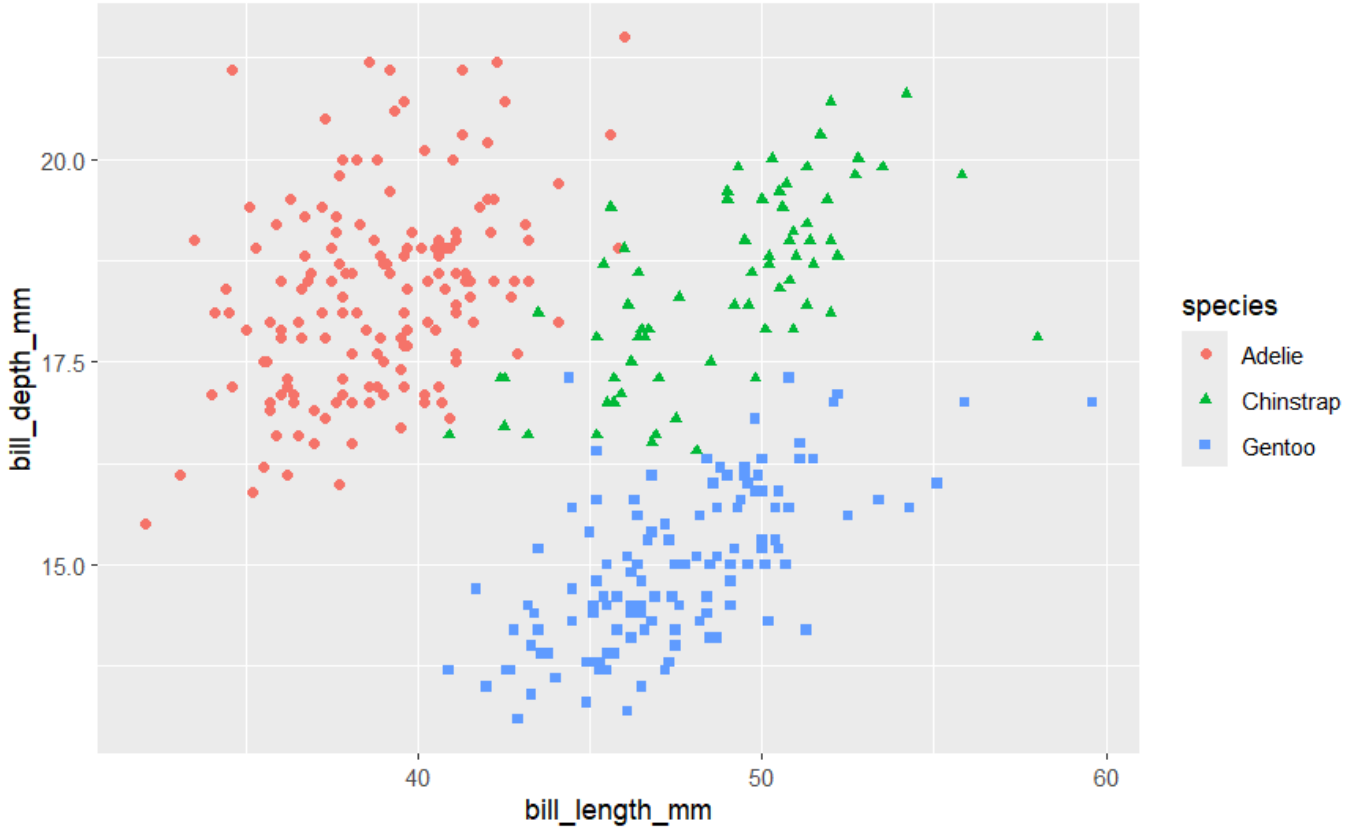


In the above graph we can see that their are two legends for the same graph inorder to remove that we need to do: shown below

labs() is majorly/mainly used for giving captions, title , subtitle to the graph along with naming of the axis. So on removing the line of code with labs() we can easily eliminate 2 legends and keep the one which is necessary.

Hide

```
ggplot(
  data = penguins,
  mapping = aes(
    x = bill_length_mm, y = bill_depth_mm,
    color = species, shape = species
  )
) +
  geom_point()
```

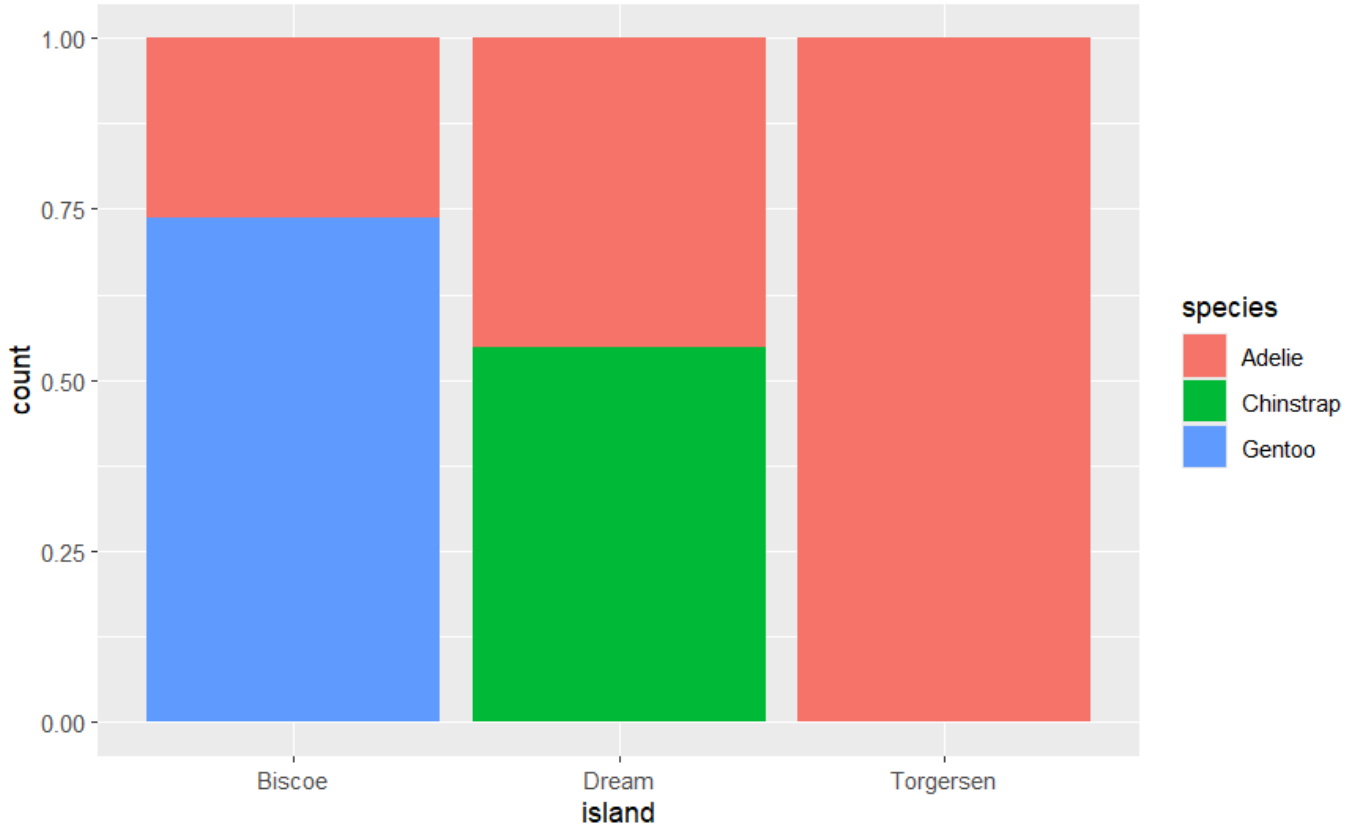


#7

What proportion of species live on which island?

Hide

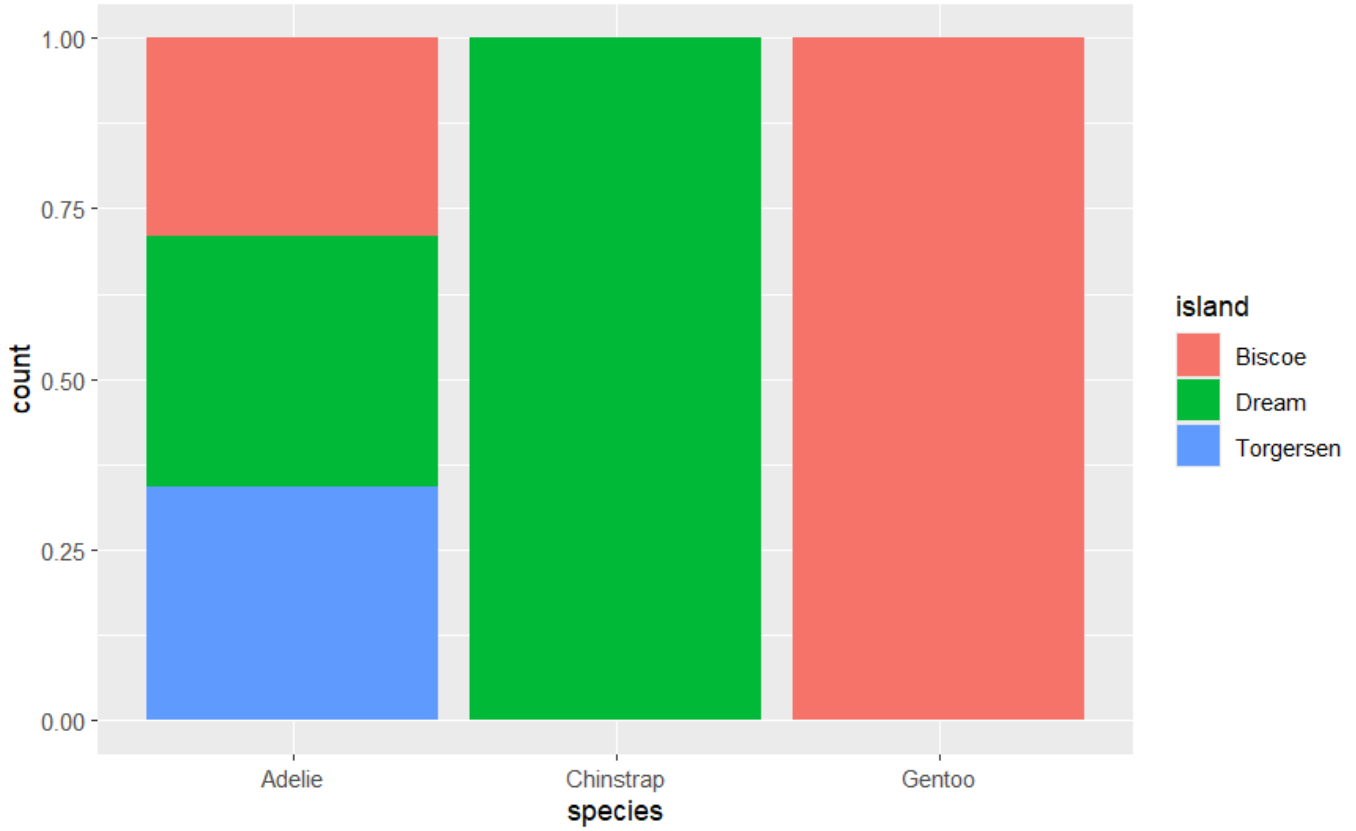
```
ggplot(penguins, aes(x = island, fill = species)) +
  geom_bar(position = "fill")
```



For the given species on which island is it found and in what proportion.

Hide

```
ggplot(penguins, aes(x = species, fill = island)) +
  geom_bar(position = "fill")
```



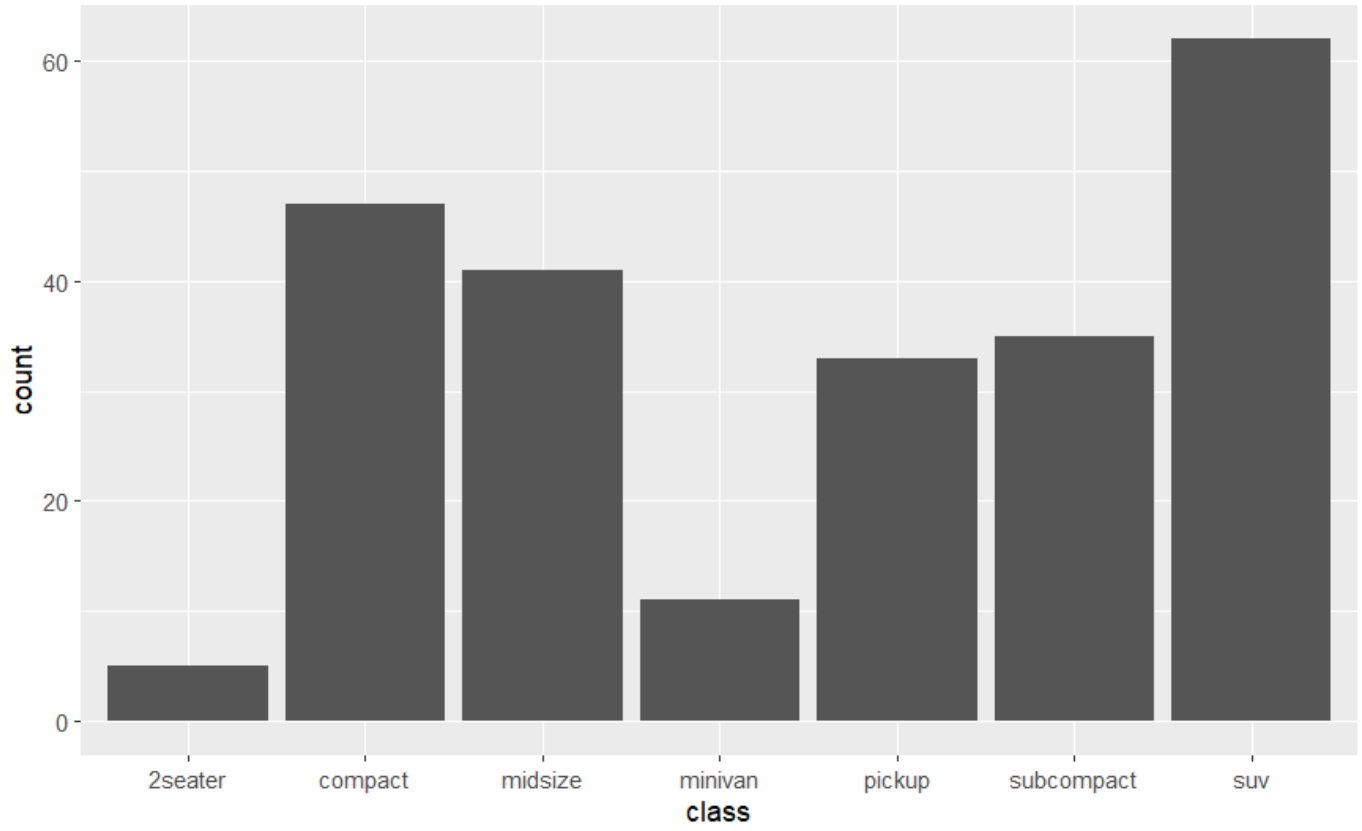
##1.6.1

#1

First the ggplot() makes the bar plot followed by the scatter plot and it saves the scatter plot as it always saves the latest (last plot made) plot made , scatter plot in this code

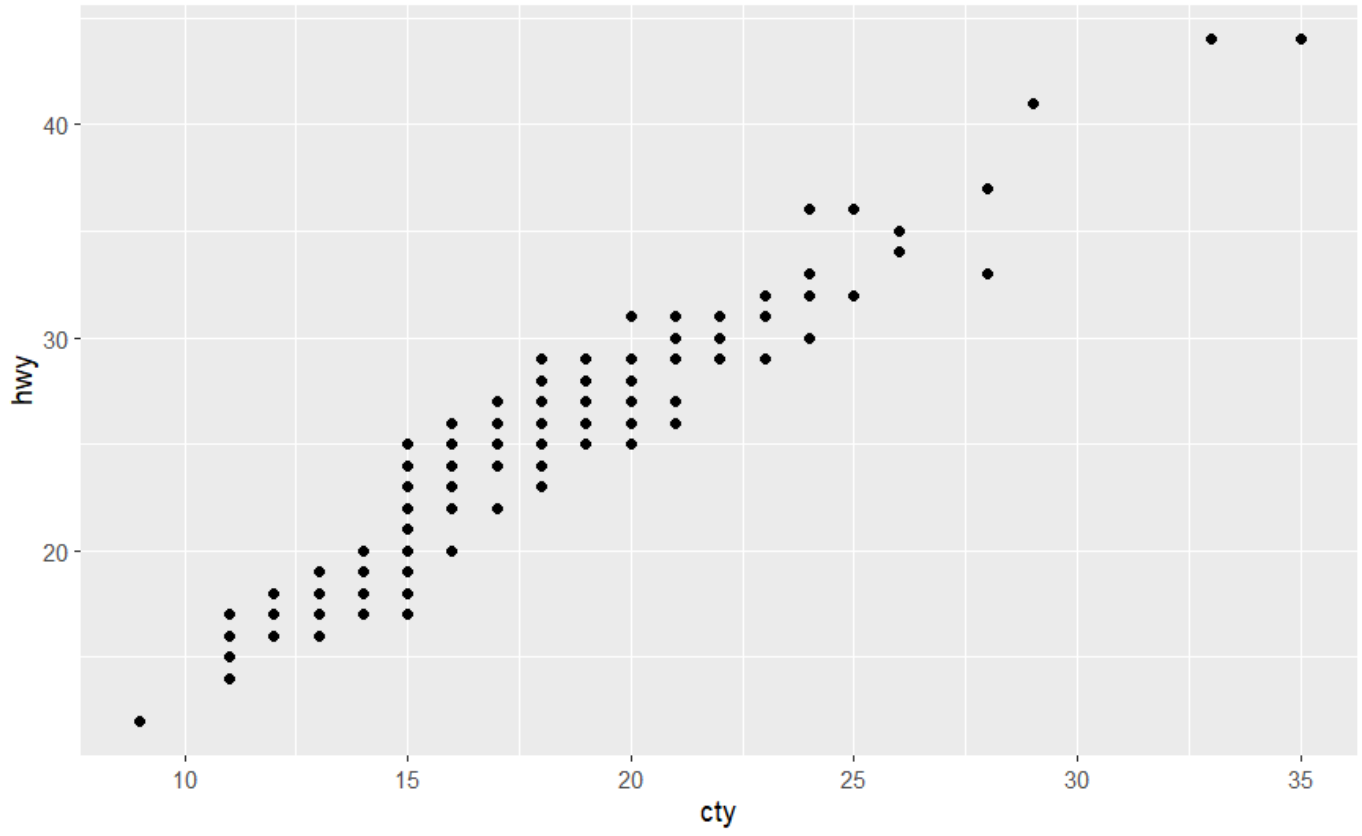
Hide

```
ggplot(mpg, aes(x = class)) +
  geom_bar()
```



Hide

```
ggplot(mpg, aes(x = cty, y = hwy)) +
  geom_point()
ggsave("mpg-plot.png")
```

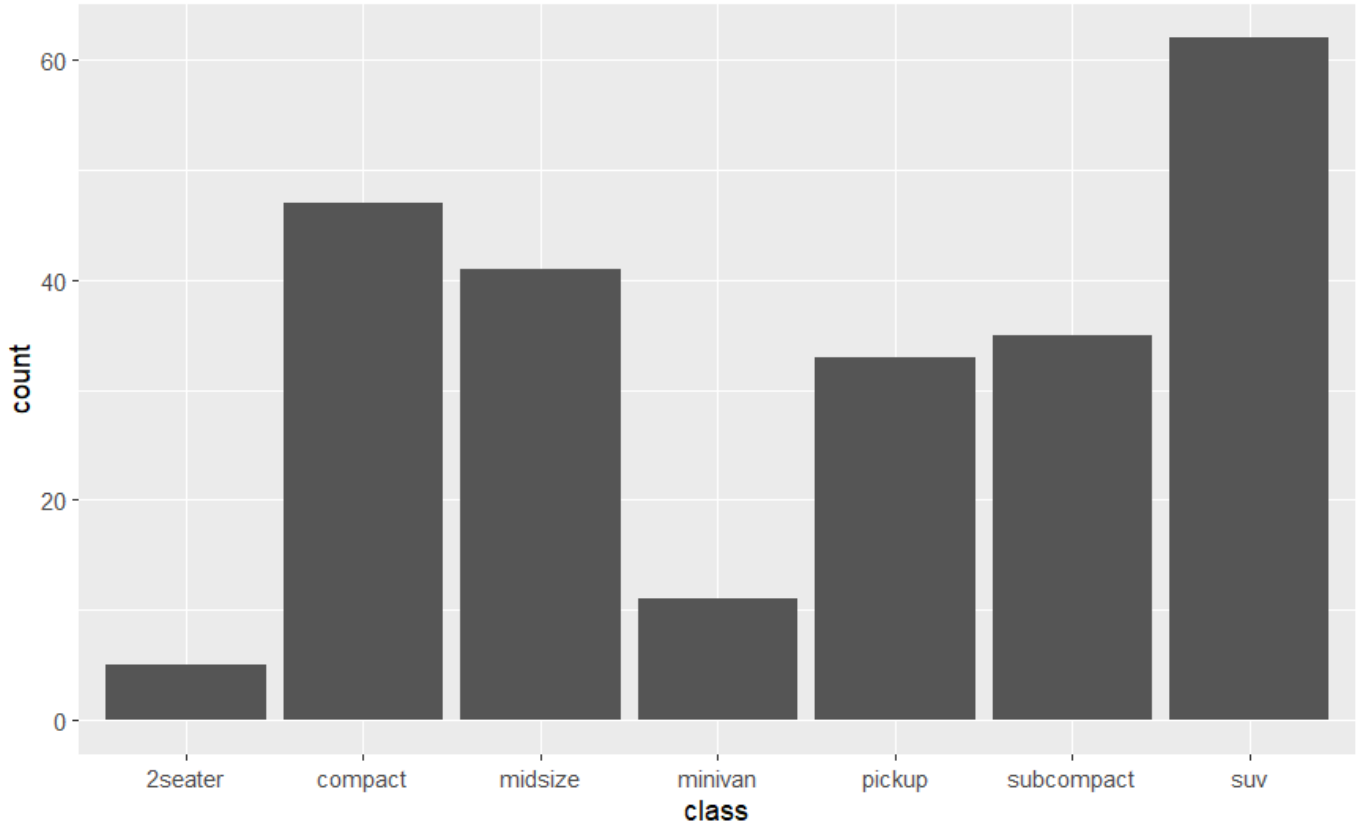


#2

Inorder for me to save my file from png to pdf I just need to change ggsave('filename.pdf') instead of .png

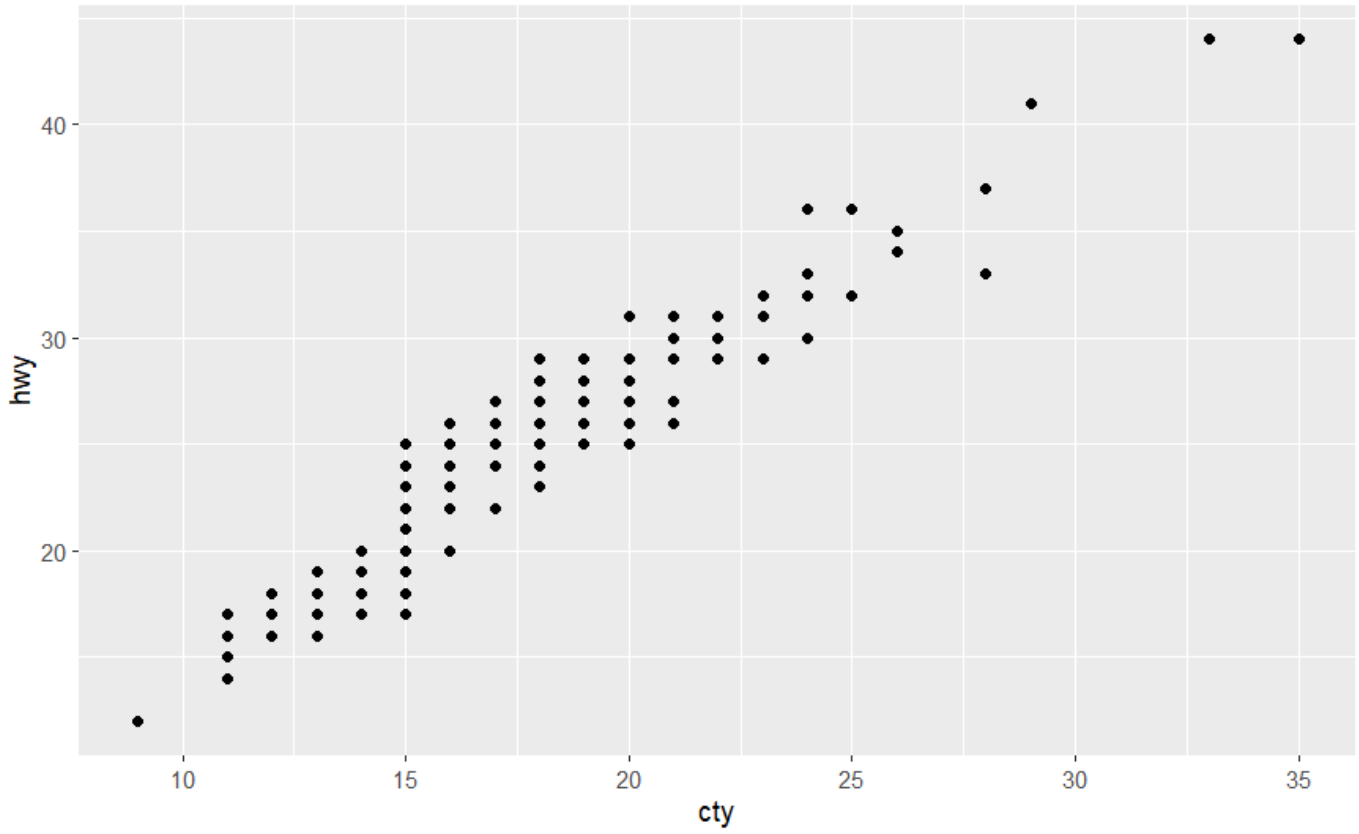
Hide

```
ggplot(mpg, aes(x = class)) +
  geom_bar()
```



Hide

```
ggplot(mpg, aes(x = cty, y = hwy)) +
  geom_point()
ggsave("mpg-plot.pdf")
```



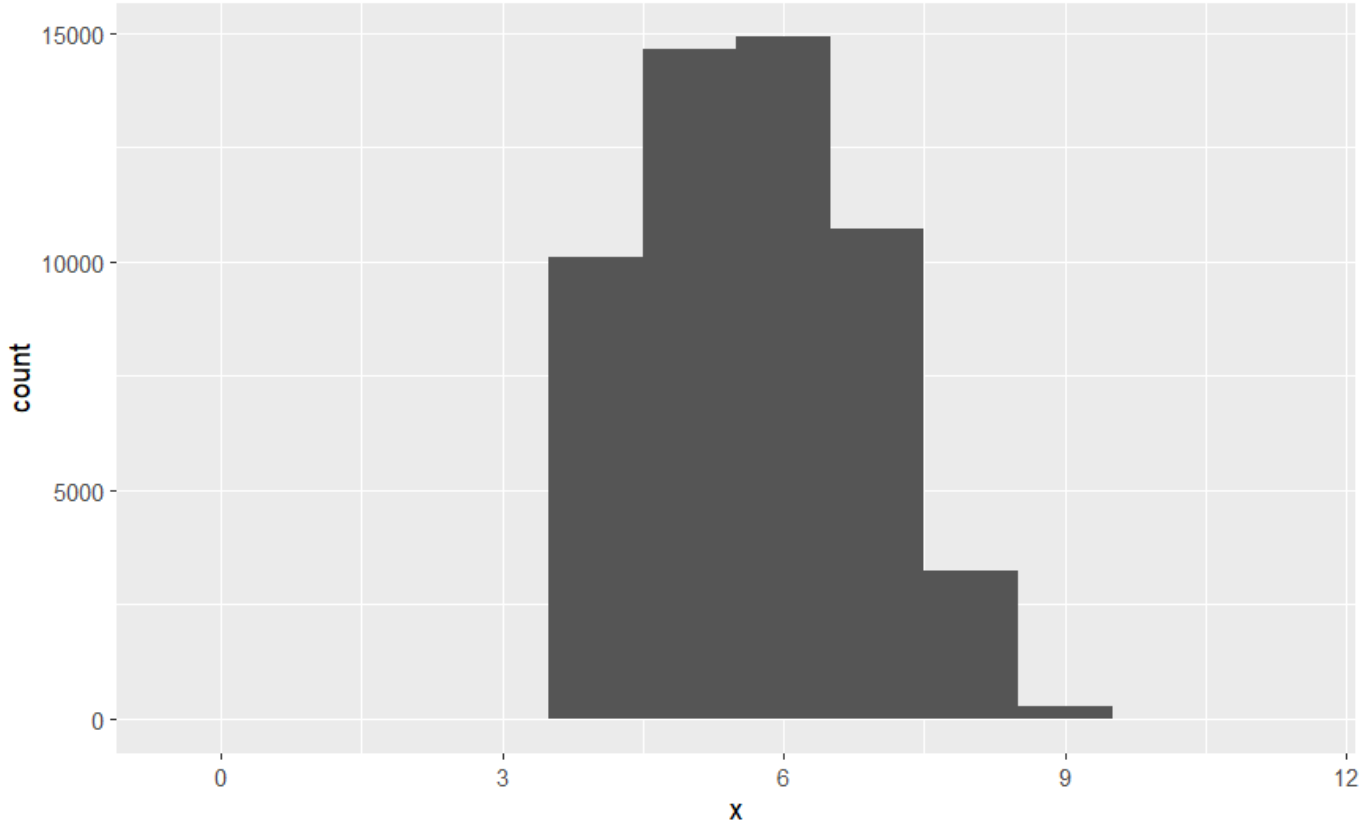
##10.3.3

#1

From what I can observe in the dataset below it is quite obvious that the z variable is the depth which has smaller vaues compared to x and y variables which makes it evident that ‘z’ variable represents the depth of diamonds,x an y have similar values (centered around 5 to 6 mm) concluding on the fact that x is the length and y is the width.

Hide

```
ggplot(diamonds, aes(x= x))+
  geom_histogram(binwidth = 1)
```

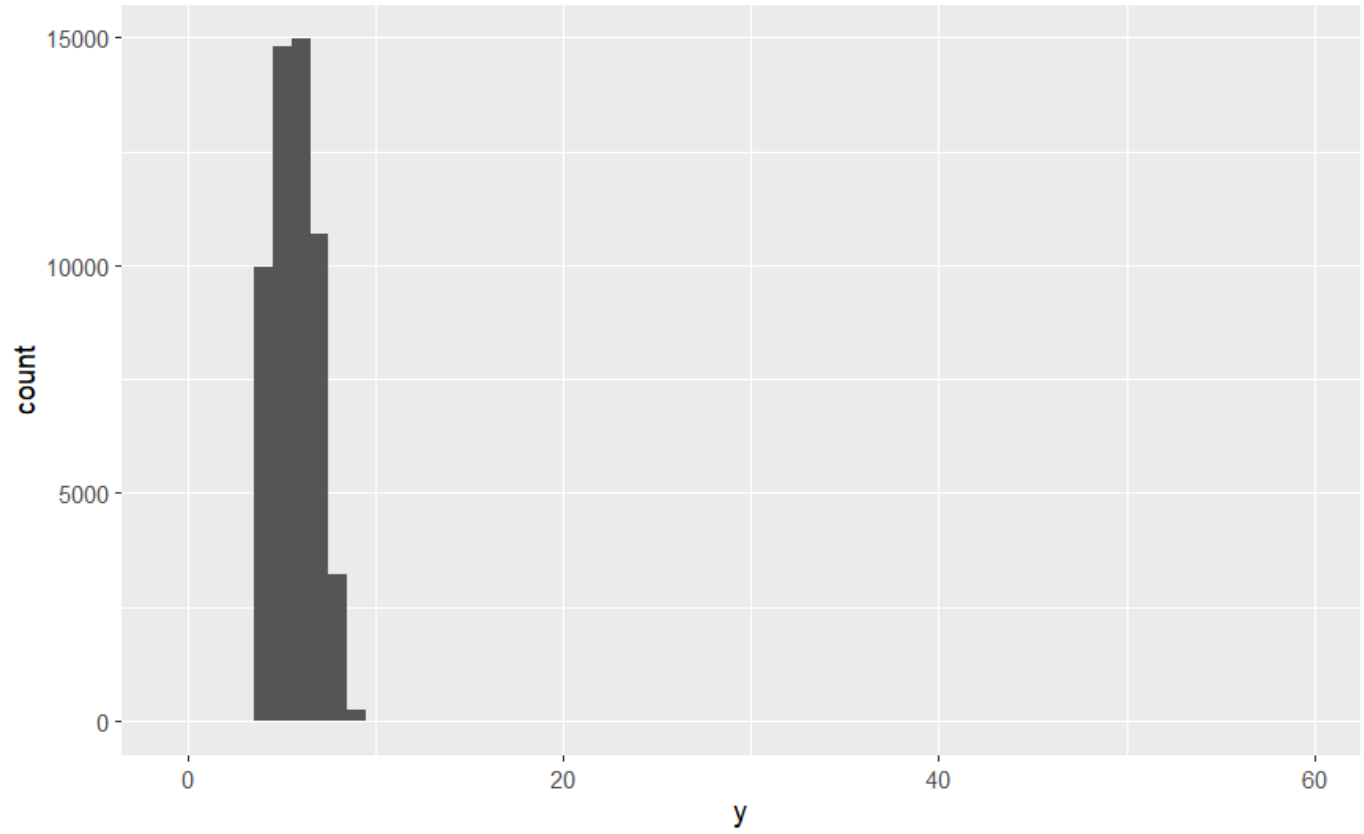


Hide

```
NA
NA
NA
```

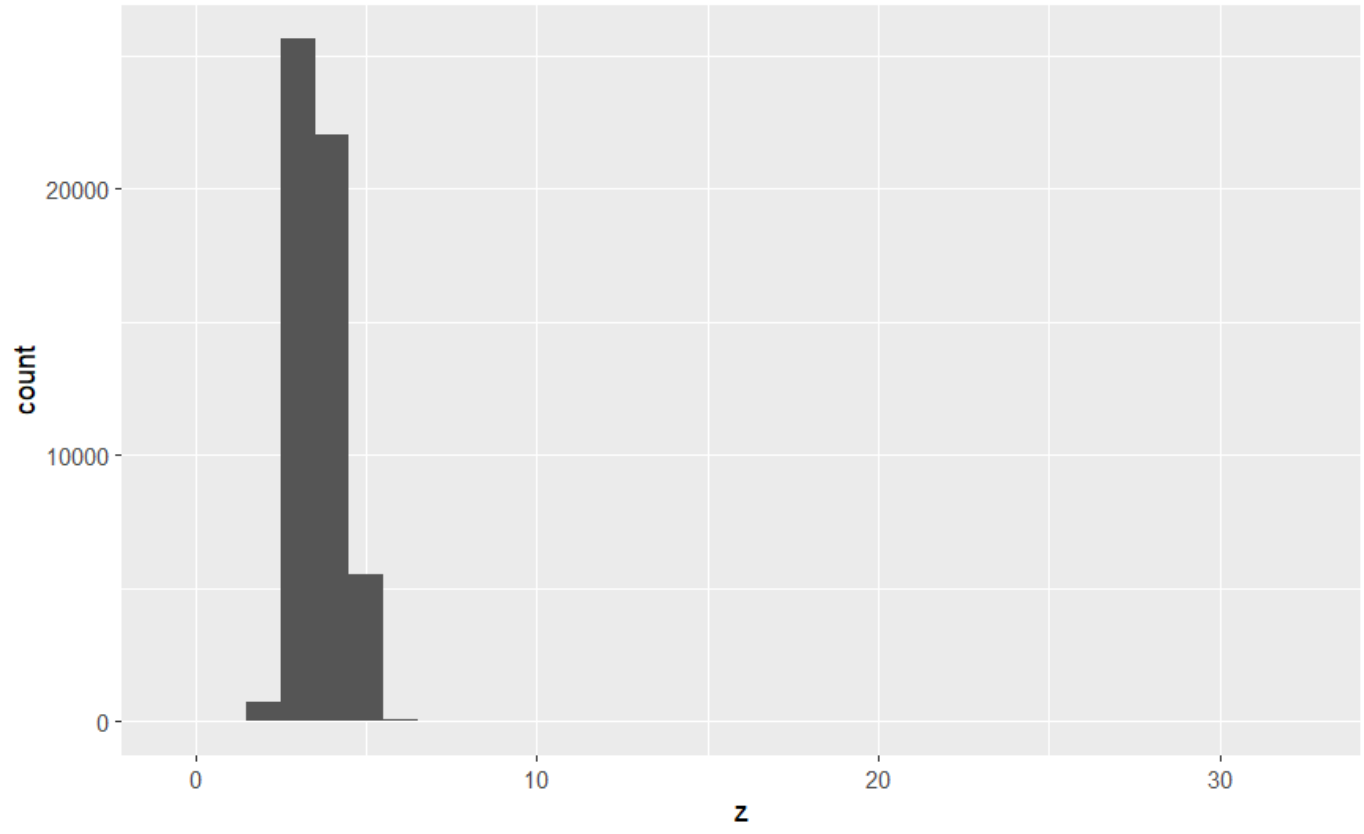
Hide

```
ggplot(diamonds, aes(x= y))+
  geom_histogram(binwidth = 1)
```



Hide

```
ggplot(diamonds, aes(x= z))+
  geom_histogram(binwidth = 1)
```



#2

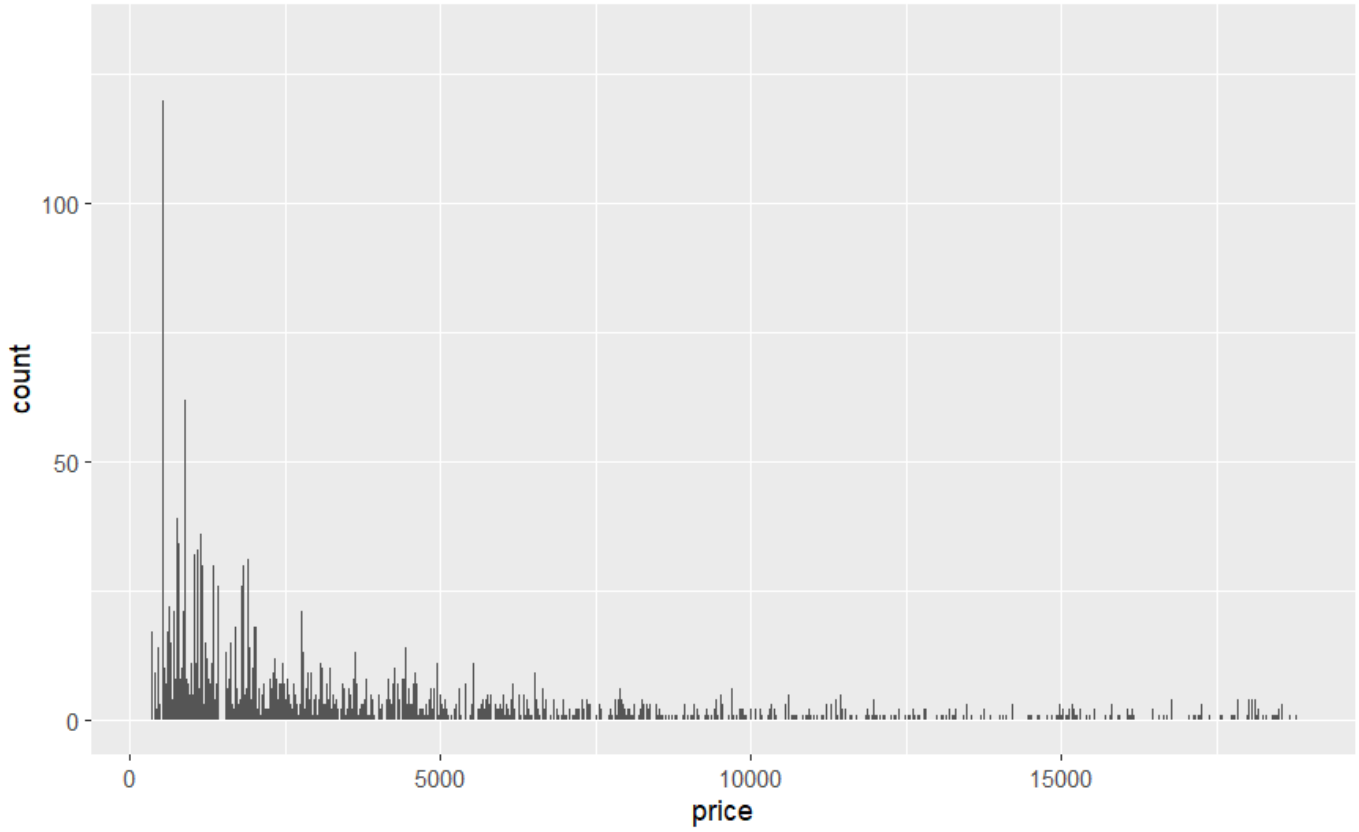
Price is highly skewed, discrete, and heaped at common price points; a log scale makes the distribution interpretable.

Small binwidths exaggerate discreteness; large binwidths hide detail — try several.

Investigate the extreme high prices: these are usually high-carat stones, so condition on carat when modeling price.

Hide

```
ggplot(diamonds, aes(price))+
  geom_histogram(binwidth = 1)
```



Hide

```
NA
NA
```

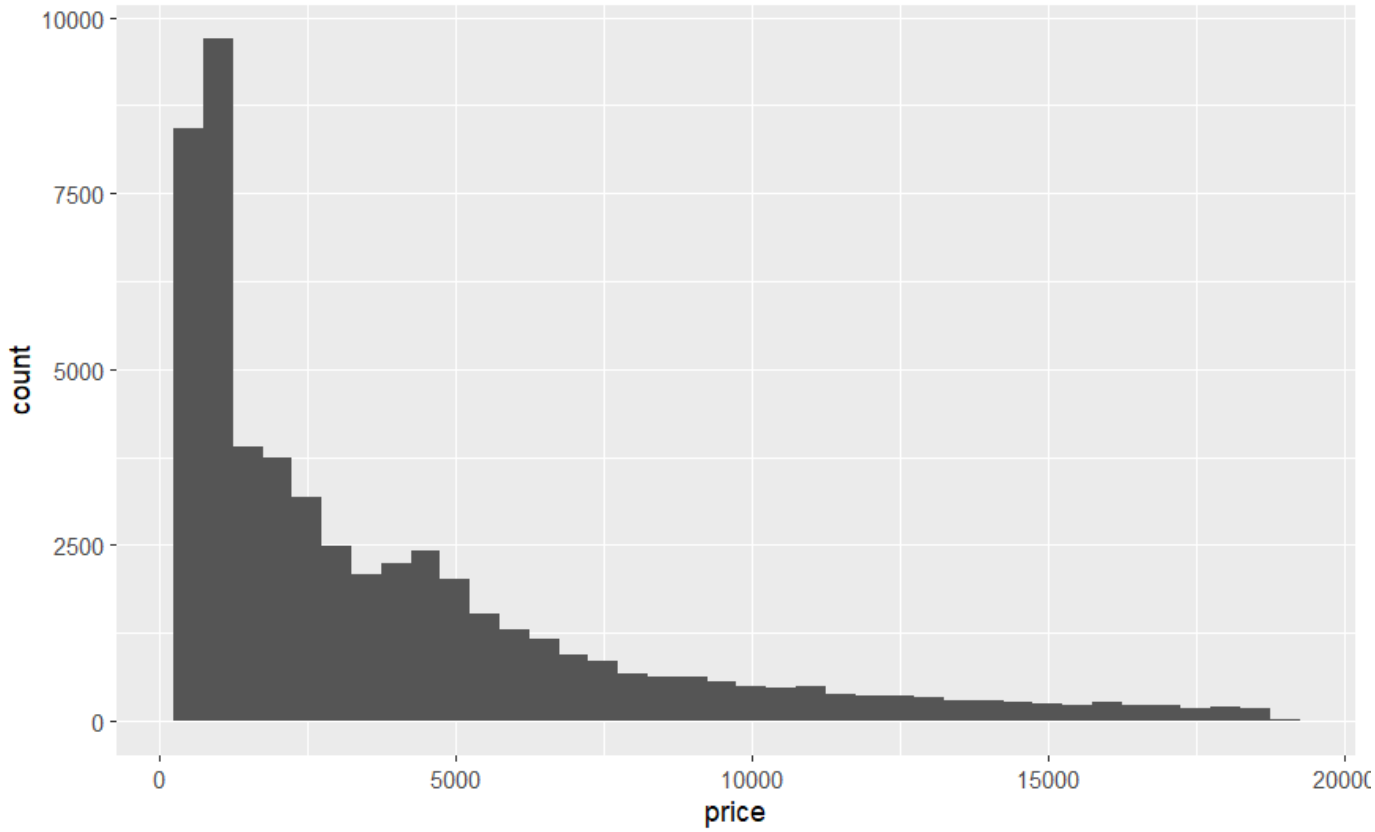
##10.5.1.1

#1

improving price distribution

Hide

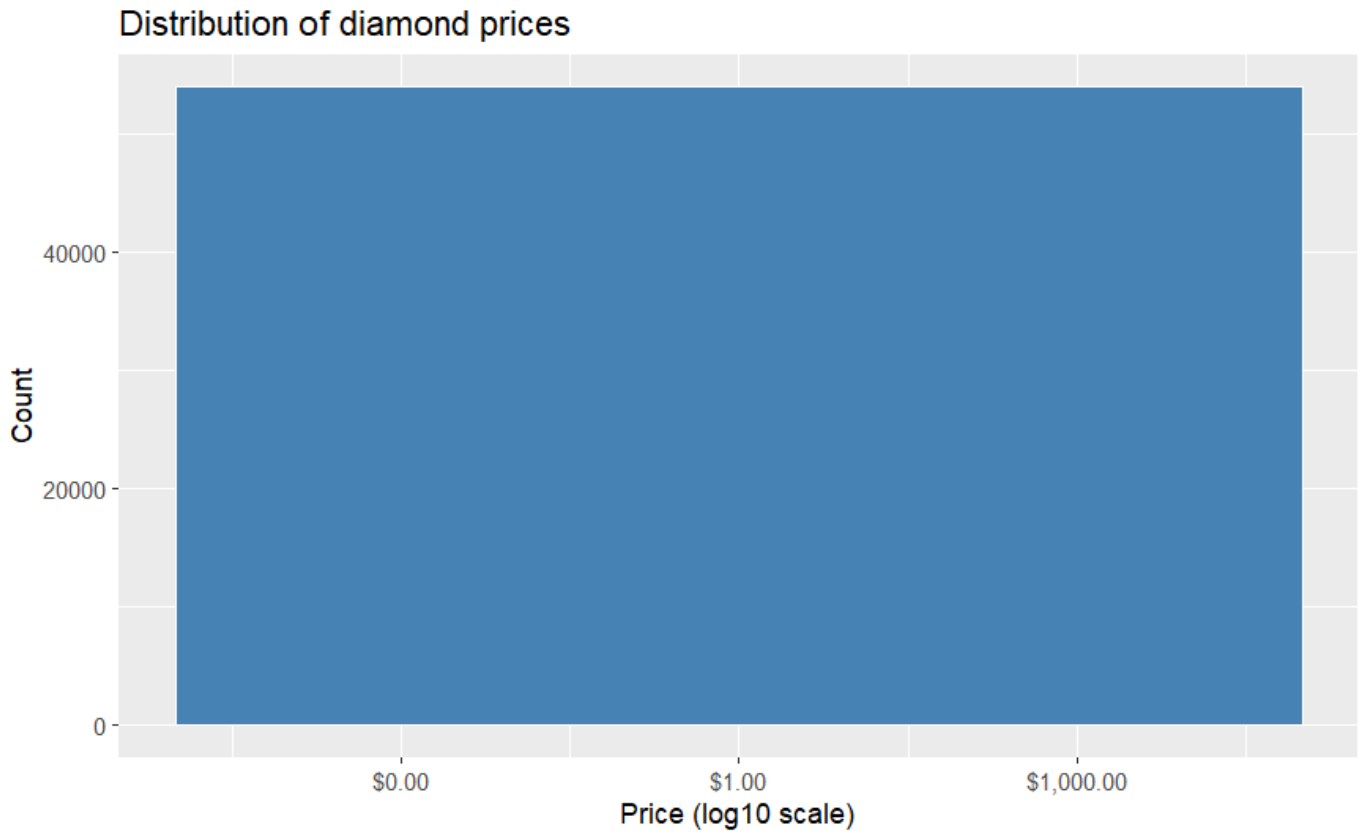
```
library(scales)
# (messy, skewed and hides structure)
ggplot(diamonds, aes(x = price)) +
  geom_histogram(binwidth = 500)
```



Hide

```
# better binning and clear comparison

ggplot(diamonds, aes(x = price)) +
  geom_histogram(binwidth = 10, fill = "steelblue", color = "white") +
  scale_x_log10(labels = dollar_format()) +
  labs(
    x = "Price (log10 scale)",
    y = "Count",
    title = "Distribution of diamond prices"
  )
```



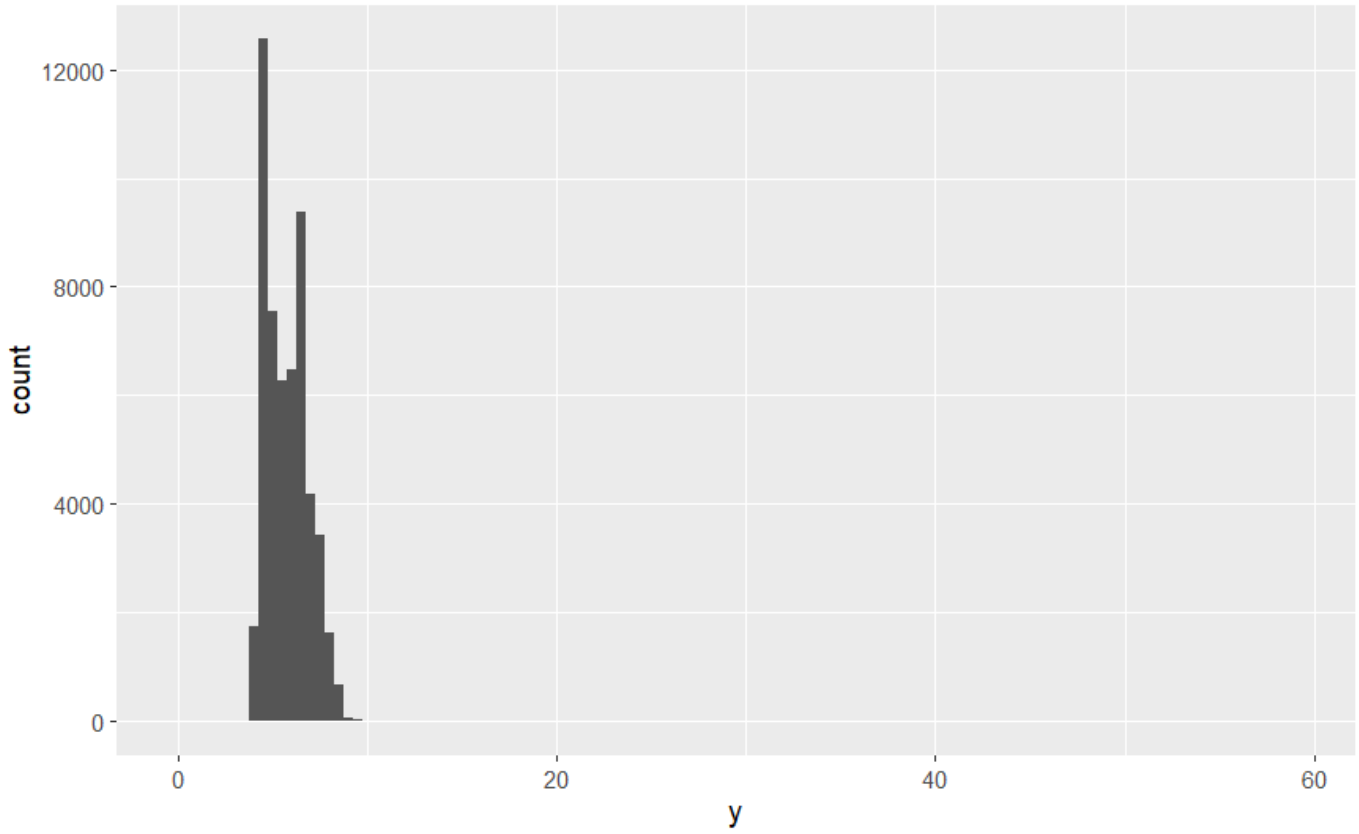
Hide

```
NA
NA
```

Hide

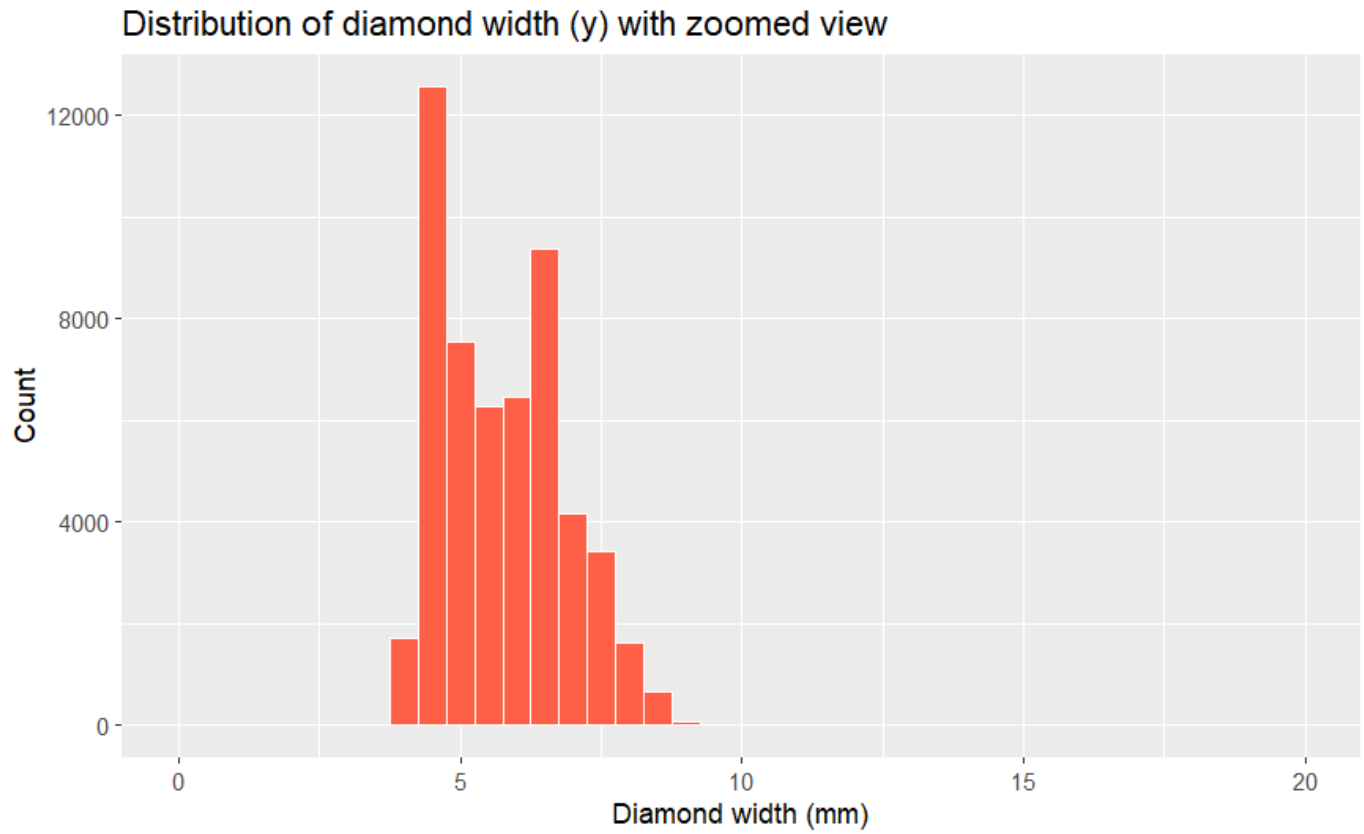
```
# zooming in with coord_cartesian to spot anomalities

ggplot(diamonds, aes(x = y)) +
  geom_histogram(binwidth = 0.5)
```



Hide

```
ggplot(diamonds, aes(x = y)) +
  geom_histogram(binwidth = 0.5, fill = "tomato", color = "white") +
  coord_cartesian(xlim = c(0, 20)) +
  labs(
    x = "Diamond width (mm)",
    y = "Count",
    title = "Distribution of diamond width (y) with zoomed view"
  )
```



Hide

```
NA
NA
NA
```

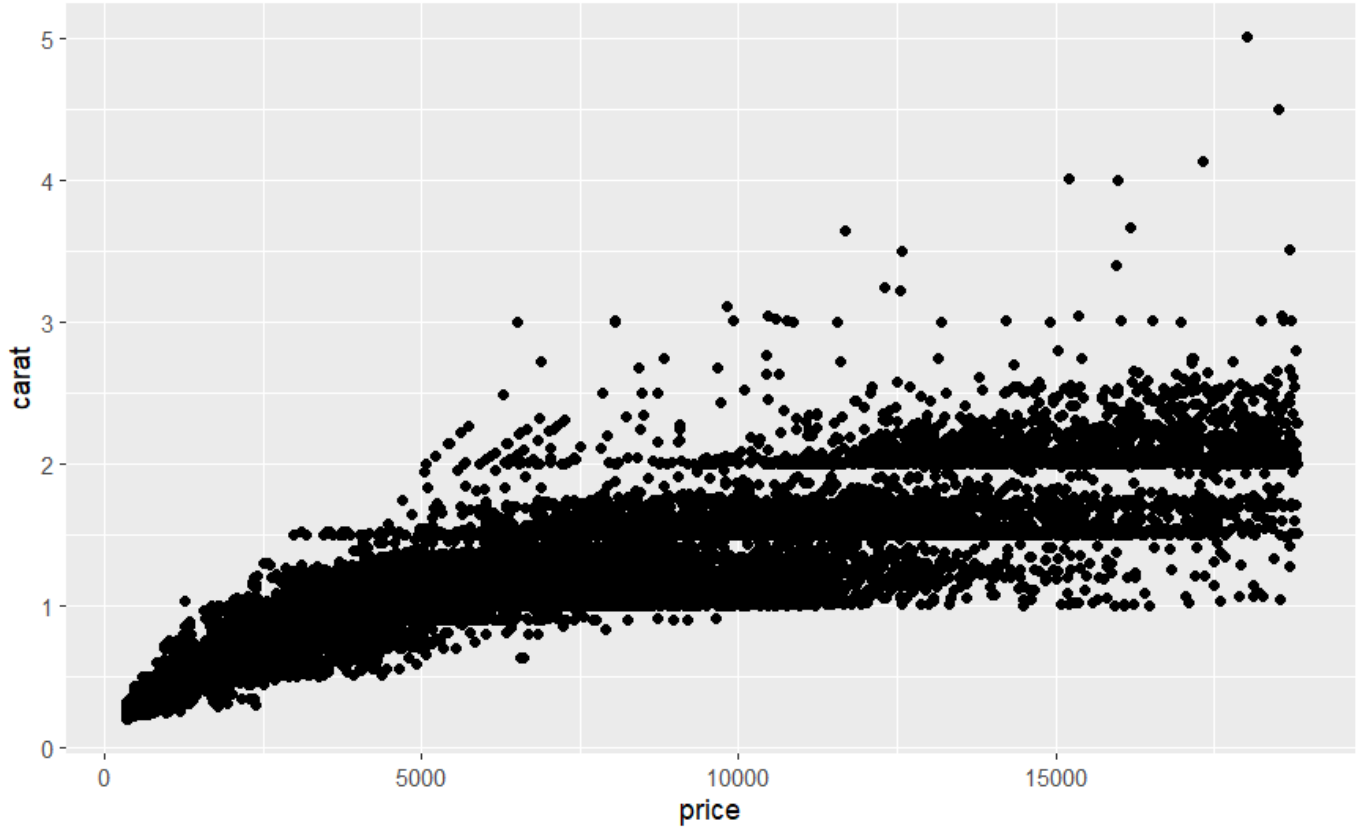
#2

–> price is most strongly related to carat.

Larger carat diamonds almost always cost more, and the relationship is very steep and nonlinear. So carat is the single most important predictor of price. We can see this below in the visualization.

Hide

```
ggplot(diamonds, aes(x = price, y= carat))+
  geom_point()
```



Hide

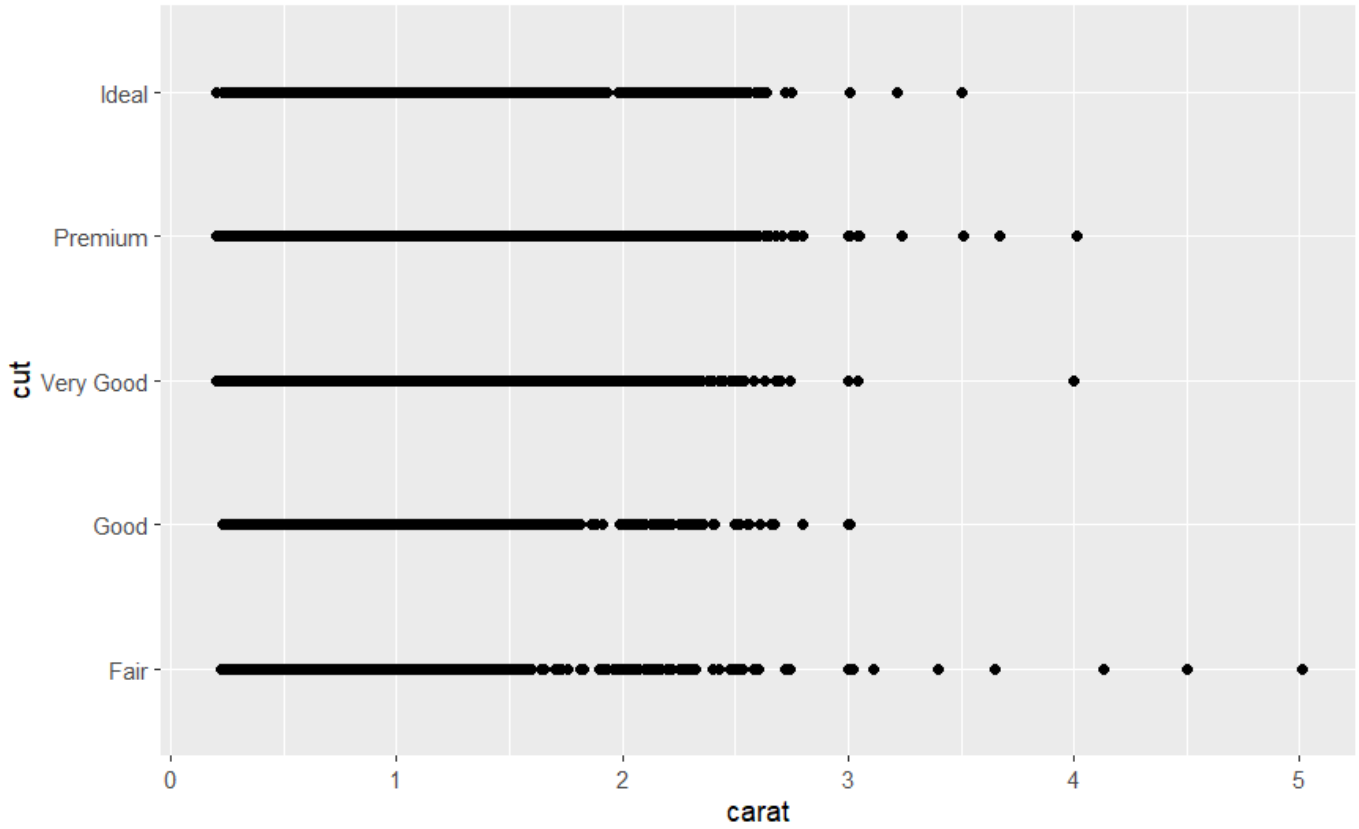
```
NA
NA
```

–> On plotting carat vs cut, you'll see that lower-quality cuts (Fair, Good) tend to have larger carat weights.

Higher-quality cuts (Very Good, Premium, Ideal) are more common at smaller carats.

Hide

```
ggplot(diamonds, aes(x= carat, y = cut))+
  geom_point()
```



–> Carat drives price: bigger diamonds cost disproportionately more.

Cut is negatively associated with carat: larger stones often have worse cut quality.

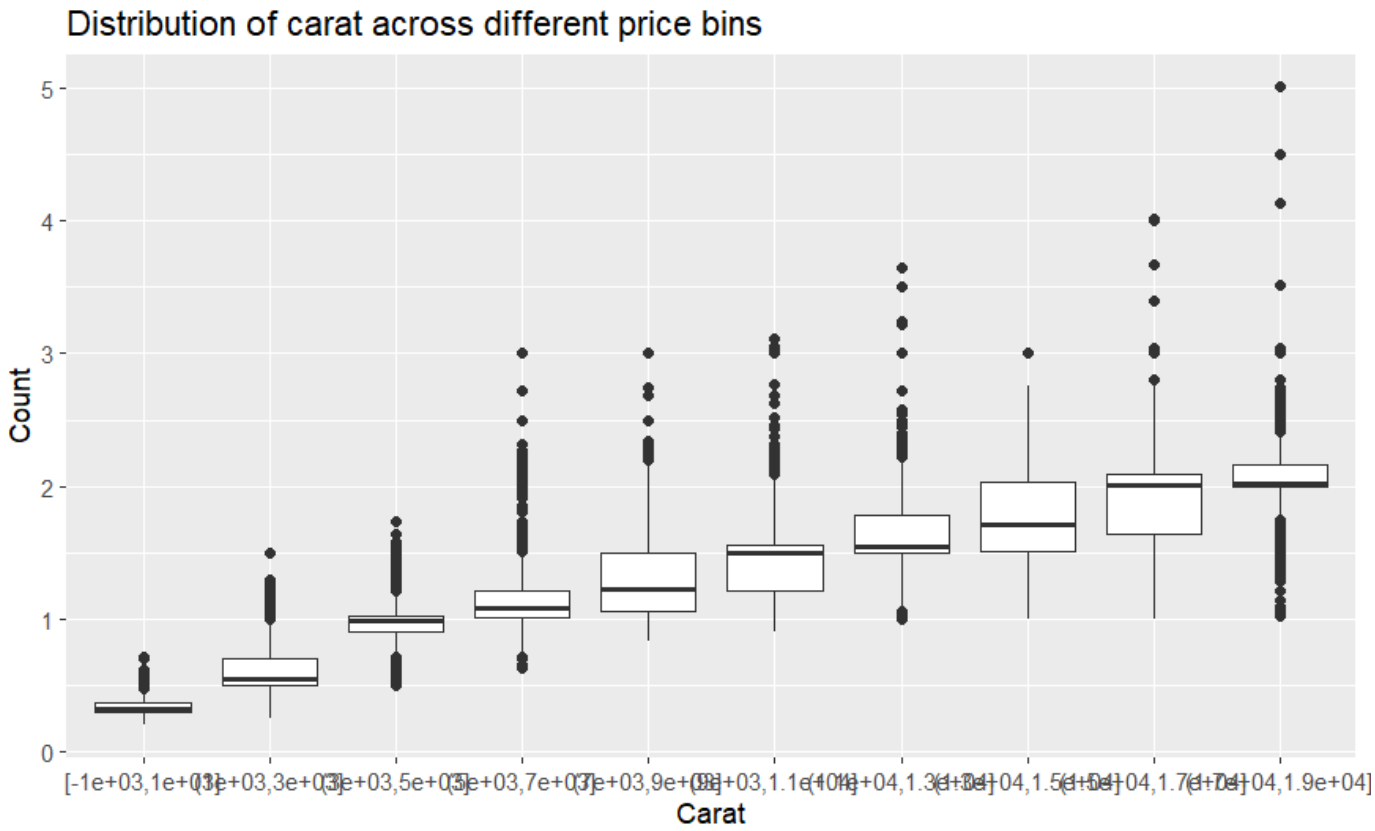
Therefore a Fair-cut diamond that is large can easily cost more than a Perfect-cut diamond that is small. The effect of carat on price outweighs the effect of cut on price.

##10.5.3

#2

Hide

```
ggplot(diamonds, aes(x= cut_width(price, 2000), y = carat))+
  geom_boxplot()+
  labs(
    x = "Carat",
    y = "Count",
    title = "Distribution of carat across different price bins"
  )
```



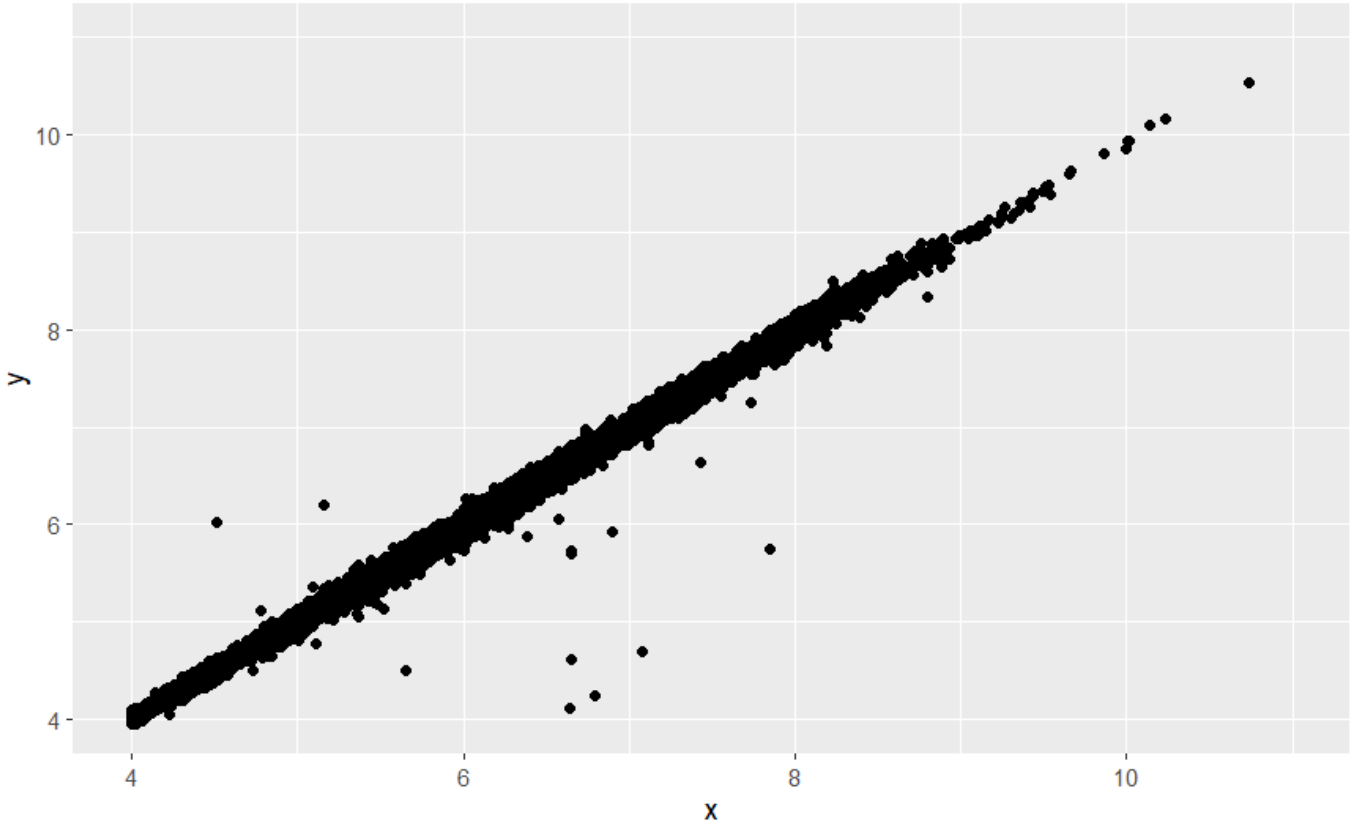
Hide

```
NA
NA
NA
NA
```

#5 A scatterplot is better than a binned plot for detecting outliers because it shows each observation individually, making unusual combinations of otherwise normal x and y values visible, whereas binned plots aggregate points and can hide those anomalies. As you can see in the plot below:

Hide

```
diamonds |>
  filter(x >= 4) |>
  ggplot(aes(x = x, y = y)) +
  geom_point() +
  coord_cartesian(xlim = c(4, 11), ylim = c(4, 11))
```

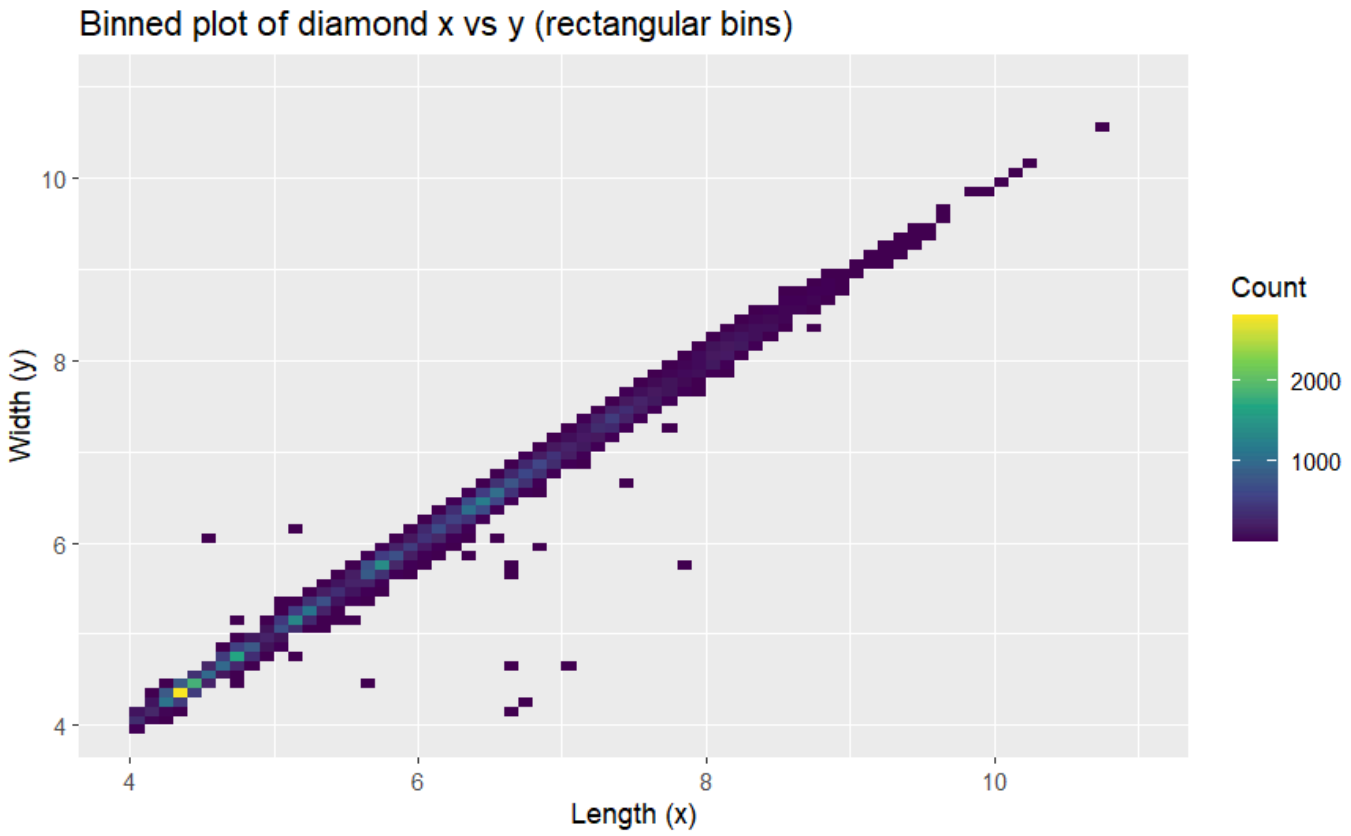


Hide

```
NA
NA
```

Hide

```
diamonds |>
  filter(x >= 4) |>
  ggplot(aes(x = x, y = y)) +
  geom_bin2d(binwidth = c(0.1, 0.1)) +
  coord_cartesian(xlim = c(4, 11), ylim = c(4, 11)) +
  scale_fill_viridis_c() +
  labs(
    x = "Length (x)",
    y = "Width (y)",
    fill = "Count",
    title = "Binned plot of diamond x vs y (rectangular bins)"
  )
```


Binned plot of diamond x vs y (rectangular bins)
Width (y)
Length (x)
Count
2000
1000
10
8
6
4
4
6
8
10

Hide

NA
NA